

Android 应用测试大作业记录

1. 测试对象与环境

- 源码目录: `D:\Codex_talk\AS\Notes-main`
- 目标 APK: `D:\notes-13-foss-release.apk`
- 目标 APK 信息: `applicationId=org.fossify.notes, versionCode=13, versionName=1.7.0, minSdk=26, targetSdk=36`
- 本地编译产物:
 - Debug: `app/build/outputs/apk/foss/debug/notes-13-foss-debug.apk`
 - Release: `app/build/outputs/apk/foss/release/notes-13-foss-release-unsigned.apk`
- 说明: 本机未配置 release 签名文件, `assembleFossRelease` 可生成未签名 release APK; 正式安装测试应使用已签名 APK 或 Android Studio 生成签名包。

使用的主要命令:

```
.\gradlew.bat :app:assembleFossDebug --console=plain --no-daemon
.\gradlew.bat :app:assembleFossRelease --console=plain --no-daemon
.\gradlew.bat :app:testFossDebugUnitTest
:app:createFossDebugUnitTestCoverageReport --console=plain --no-daemon
```

覆盖率报告位置:

```
app/build/reports/coverage/test/foss/debug/index.html
app/build/reports/coverage/test/foss/debug/report.xml
```

2. 测试配置调整

在 `app/build.gradle.kts` 中为 debug 构建开启了覆盖率:

- `enableUnitTestCoverage = true`
- `enableAndroidTestCoverage = true`
- `unitTests.isIncludeAndroidResources = true`

新增测试依赖:

- JUnit 4: 基础 JVM 单元测试
- MockK: 模拟 `Context`、`ContentResolver`、`Uri` 等协作对象
- Robolectric: 在 JVM 上运行 Android 资源和控件相关单元测试
- AndroidX Test Core: 提供 `ApplicationProvider`
- Jacoco: 为 Robolectric/无源码位置类启用覆盖率采集兼容配置

3. 三阶段测试与覆盖率变化

阶段	新增测试	测试数量	指令覆盖率	分支覆盖率	行覆盖
阶段一：JUnit 基础单元测试	<code>NoteTypeConverterTest</code> 、 <code>StringChecklistTest</code> 、 <code>CollatorBasedComparatorTest</code>	10	1%	1%	68 / 5211
阶段二：MockK 测试	<code>NoteMockKTest</code>	14	1%	2%	86 / 5211
阶段三： Robolectric 安卓单元测试	<code>RobolectricAndroidExtensionsTest</code>	19	2%	2%	110 / 5211

低的核心原因：覆盖率报告的分母是整个 fossDebug App，不是只算你新增测试涉及的几个类。

这次 19 个测试主要覆盖的是：

`NoteTypeConverter` `checklist` 字符串转换 `CollatorBasedComparator` `Note.getNoteStoredValue` `Config / TextView` 的少量 Android 环境行为 但整个 App 里有大量还没被测试跑到的代码：

`MainActivity` 很大，几乎没被单测触发 `SettingsActivity`、`Fragment`、`Dialog`、`Adapter` 都没怎么覆盖 `DataBinding` 生成类也进入了覆盖率统计 数据库、`Widget`、导入导出、权限、文件选择、菜单交互等流程还没测 当前是 JVM 单元测试为主，不是完整 GUI/instrumentation 测试 所以总覆盖率看起来低：最终是 110 / 5211 行，也就是约 2%。但这不代表测试没生效，局部效果其实明显：

`Note.getNoteStoredValue(Context)`：指令覆盖 89%，分支覆盖 80% `Note` 类：指令覆盖 65%

`CollatorBasedComparator`：指令覆盖 98% `TextViewKt.maybeRequestIncognito()`：100% 也就是说，局部核心函数覆盖不错，但项目总体太大，测试覆盖面还窄。

阶段一：JUnit

目标是先覆盖不依赖 Android 运行时的纯逻辑：

- `NoteType.fromValue()` 和 `NoteTypeConverter`
- 清单 JSON 与纯文本转换：`parseChecklistItems()`、`checklistToPlainText()`
- 自然排序比较器：`CollatorBasedComparator`

阶段一建立了基础覆盖率，适合作为后续 Mock 和 Android 单元测试的对照。

阶段二：MockK

目标是覆盖带 Android 协作对象的业务分支，但仍然保持 JVM 单元测试速度：

- 空 `path` 时直接返回 `Note.value`
- 普通文件路径时读取本地文件内容
- `content://` 路径时通过 mocked `ContentResolver` 读取输入流
- 外部内容读取异常时返回 `null`

阶段二后，模型层覆盖明显提升：

- `org.fossify.notes.models` 包指令覆盖率提升到 43%
- `Note` 类指令覆盖率提升到 65%
- `Note.getNoteStoredValue(Context)` 方法指令覆盖率达到 89%，分支覆盖率达到 80%

阶段三：Robolectric

目标是在不启动虚拟机的情况下，把 Android 资源、`Context`、`TextView` 和 `SharedPreferences-backed` 配置纳入单元测试：

- 使用 `ApplicationProvider.getApplicationContext()` 读取变体资源
- 验证 `Config` 中字体比例、文本对齐配置映射
- 验证 `TextView.maybeRequestIncognito()` 对 `IME_FLAG_NO_PERSONALIZED_LEARNING` 的开启和移除

阶段三后，覆盖率继续提升：

- 总行覆盖从 86 行提升到 110 行
- `org.fossify.notes.helpers` 包指令覆盖率提升到 18%
- `Config` 类指令覆盖率达到 17%，分支覆盖率达到 33%
- `TextViewKt.maybeRequestIncognito(TextView)` 达到 100% 指令覆盖和 100% 分支覆盖

4. 最终验证结果

最终执行结果：

- `:app:testFossDebugUnitTest`：19 个测试全部通过，0 failure，0 error
- `:app:createFossDebugUnitTestCoverageReport`：成功生成 HTML/XML 覆盖率报告
- `:app:assembleFossDebug`：成功
- `:app:assembleFossRelease`：成功，生成未签名 APK

最终产物：

```
D:\Codex_talk\AS\Notes-main\app\build\outputs\apk\foss\debug\notes-13-foss-debug.apk
D:\Codex_talk\AS\Notes-main\app\build\outputs\apk\foss\release\notes-13-foss-release-unsigned.apk
```

5. 结论

本次测试按照三个阶段递进：

1. JUnit 覆盖纯 Kotlin 逻辑，建立初始覆盖率。
2. MockK 覆盖需要外部协作对象的分支，提升模型层覆盖率。
3. Robolectric 覆盖 Android 资源、`Context`、`TextView` 和配置行为，进一步提升 JVM 环境下的 Android 单元测试能力。

覆盖率从阶段一的 1% / 1% / 68 行，提升到最终阶段的 2% / 2% / 110 行。由于项目 UI、DataBinding、Activity、Dialog 和 Adapter 代码量较大，当前覆盖率百分比仍然较低；后续若要继续提高，应优先补充 `NotesHelper`、数据库 DAO、Activity/Fragment 关键流程和 GUI 自动化测试。